Lab0 GitLab 实验报告

一. 思考题

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我曾在《人工智能导论》课程当中有过三人小组协同开发的经历，在该门课程中首次体验了利用 github 协同开发，每人实现期末 project 中的某一部分，高效完成了任务。

2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

暂存是存在本地，而提交是将仓库中也进行同步，这样的设计能够方便开发者针对某一模块做整体的修改后一并提交；且这样的设计将一次提交中的每个步骤能够区分开，在 debug 时更加条理清晰；这样的设计使得每一次提交既有完整性，又有完整性。

3. git branch 和 git branch -a 的区别是什么？查阅资料并回答。

git branch 只列出本地分支，而 git branch -a 会同时列出本地分支和远程跟踪分支（remotes/origin/...），前者适合日常查看和切换本地分支，后者适合了解远程仓库有哪些分支。

二．网页阅读讨论

阮一峰的《Commit message 和 Change log 编写指南》介绍了 Angular 提交信息规范，主张用 type(scope): subject 的格式让提交历史更清晰、可过滤，并能自动生成 Change log，同时推荐了 Commitizen 和 validate-commit-msg 等工具来落地这套规范。Vectania 的《Gitflow 使用规范》则梳理了 master、develop、feature、release、hotfix 五类分支的职责与流转关系，强调从哪来、回哪去，用临时分支隔离功能开发与缺陷修复，保证主线稳定。综合这两篇来看，学习 Git 不只是学会敲命令，而是掌握一套协作语言和工作纪律：规范的提交信息让历史可读、可查、可自动生成发布说明，清晰的分支策略让多人并行开发时互不干扰、线上版本始终可控。归根结底，Git 能力直接决定团队协作的效率和代码交付的可靠性，是现代软件开发中绕不开的基本功。

三．主要实验步骤

1.对 main.c 中的 TODO 下代码进行了修改，并完成暂存与提交

```c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("hello ics\n");      你，4小时前 • 对于TODO的修改
    printf("Hello, world!\n");
}
```

2.刻意制造了冲突并完成合并



